

UITS

Future will be better than thy past

An initiative of PHP Family

University of Information Technology & Sciences

Project Report

On`

ONLINE CRIME REPORTING AND CASE MANAGEMENT SYSTEM

Course Code: CSE0613226

Course Title: Software Engineering and System Analysis Lab

Team: Syntax Squad

Submitted by

Name: Md Nayem Islam
ID: 04325105101041
Department: CSE

Name: Md Zayed Adnan Zabin
ID: 04325105101034
Department: CSE

Name: Mahjabin Afroze Mahira
ID: 04325105101007
Department: CSE

Name: Kanis Akter
ID: 04325105101018
Department: CSE

Submitted to

Shad Ahmed
Lecturer
Dept. of CSE, UITS

Md. Faysal
Lecturer
Dept. of CSE, UITS

Date: 14/11/2026

ABSTRACT

The Online Crime Reporting and Case Management System (CRMS) is a web-based platform that makes crime reporting and case handling easier, faster and more transparent. The primary objective is to replace manual paperwork with a centralized system in which Citizens, Police Officers and Administrators each work through their own secure portal. Citizens can register, submit crime reports with supporting images or PDF evidence, track the status of their reports through a timeline and receive in-system notifications. Police Officers can search and filter reports, verify or reject them with remarks, and manage investigations through the Investigating, Resolved and Closed stages. Administrators manage citizen and police accounts and crime categories and monitor the system through a dashboard with status and category charts.

Every report receives a unique Case ID (CR-YYYY-NNNNN) and every status change is recorded in a case status history. The system enforces role-based access control, CSRF protection, prepared statements, output escaping and validated file uploads. It is developed using PHP, MySQL, HTML5, CSS3, Bootstrap, JavaScript and runs on a local XAMPP server. The project was documented through a Software Requirements Specification, UML and data models, and verified with 136 unit and integration test cases; the five defects found were fixed and all tests passed on re-run.

TABLE OF CONTENTS

ABSTRACT

LIST OF FIGURES

CHAPTER 1 (INTRODUCTION)

- 1.1 Project Overview
- 1.2 Project Purpose & Goals
- 1.3 Project Scope
- 1.4 Stakeholders
- 1.5 Project Schedule

CHAPTER 2 (SOFTWARE REQUIREMENT SPECIFICATION)

- 2.1 Functional Requirements
- 2.2 Non-Functional Requirements
- 2.3 Business Rules
- 2.4 Operating Environment and Constraints
- 2.5 Data Requirements
- 2.6 Field Layouts and Validation Rules
- 2.7 Requirement Traceability Matrix

CHAPTER 3 (SYSTEM ANALYSIS & DESIGN)

- 3.1 Use Case Diagram
- 3.2 Data Flow Diagrams
- 3.3 Entity Relationship Diagram (ERD)
- 3.4 Class Diagram
- 3.5 Sequence Diagram
- 3.6 State Diagram

CHAPTER 4 (DATABASE DESIGN)

- 4.1 Database Schema Overview
- 4.2 Table Descriptions
- 4.3 Database Relationships

CHAPTER 5 (SYSTEM ARCHITECTURE & TECHNOLOGY STACK)

- 5.1 Architecture Overview
- 5.2 Technology Stack
- 5.3 Development and Testing Tools
- 5.4 Security Architecture

CHAPTER 6 (IMPLEMENTATION)

- 6.1 System Modules
- 6.2 Core Helper Functions
- 6.3 Key Features Implemented
- 6.4 Differences Between the SRS and the Final Implementation

CHAPTER 7 (USER INTERFACE)

- 7.1 Interface Design Principles
- 7.2 Main User Interface Screens

CHAPTER 8 (SOFTWARE TESTING)

8.1 Testing Objective and Environment
8.2 Test Strategy
8.3 Test Execution Summar
8.4 Coverage Summary
8.5 Defects Found and Fixed
8.6 Selected Test Cases
8.7 Defect Categories Not Covered by the Current Tests

CHAPTER 9 (CONCLUSION & FUTURE WORK)

9.1 Project Summary
9.2 Key Accomplishments
9.3 System Benefits
9.4 Future Enhancements
9.5 Recommendations
9.6 Final Remarks

REFERENCES

JUSTIFICATION FORM

LIST OF FIGURES

Figure 1: Use Case Diagram of the Online Crime Reporting and Case Management System
Figure 2: Citizen Use Case Diagram
Figure 3: Police Officer Use Case Diagram
Figure 4: Administrator Use Case Diagram
Figure 5: Level 0 DFD (Context Diagram)
Figure 6: Level 1 DFD
Figure 7: Entity Relationship Diagram
Figure 8: Class Diagram
Figure 9: Sequence Diagram: Report Submission, Verification and Investigation Update
Figure 10: State Diagram of a Crime Report
Figure 11: System Architecture Diagram
Figure 12: Login Screen
Figure 13: Citizen Registration Screen
Figure 14: Citizen Dashboard (new report pending)
Figure 15: Citizen Notifications
Figure 16: Report Tracking with Status Timeline (closed case)
Figure 17: Citizen Dashboard after the Report is Closed
Figure 18: Help & Resources Page
Figure 19: Police Officer Dashboard
Figure 20: Verify or Reject Incident Report
Figure 21: Report Details after Verification
Figure 22: Closed Case with Investigation History
Figure 23: Administrator Executive Dashboard
Figure 24: Manage Police Officers
Figure 25: System-Wide Crime Reports

CHAPTER 1
INTRODUCTION

1.1 Project Overview

The Online Crime Reporting and Case Management System (CRMS) is a web-based application designed to make the process of reporting and managing crimes easier, faster and more organized. Instead of relying on manual paperwork or spreadsheets, the system gives Citizens, Police Officers and Administrators one centralized platform where each role can carry out its own tasks. Citizens can report incidents online, attach supporting evidence and follow the progress of their case. Police Officers can review, verify or reject reports and record investigation progress. Administrators manage user accounts and crime categories and monitor overall activity through a dashboard.

Every report receives a unique Case ID in the format CR-YYYY-NNNNN and moves through a defined lifecycle: Pending, Verified, Investigating, Resolved and Closed (or Rejected at the verification stage). Each step creates an in-system notification for the citizen so that the case remains transparent from submission to closure.

Technical Perspective

The system follows a three-tier architecture. PHP (with PDO and session-based authentication) provides the application logic, MySQL stores all data, and the front end is built with HTML5, CSS3, Bootstrap and JavaScript. The dashboards use Chart.js for status and category charts. The application runs on a local XAMPP server (Apache, PHP, MySQL) and was developed with Visual Studio Code, using Git and GitHub for version control.

Domain Perspective

Crime reporting is a trust-sensitive public-safety service. Citizens need a simple way to report incidents and know what is happening to their report, while police need an organized queue of cases to triage. CRMS addresses this by combining role-based access control, validated evidence upload, a structured verification and investigation workflow, and a complete case status history.

1.2 Project Purpose & Goals

1.2.1 Purpose

The purpose of CRMS is to simplify crime reporting and case handling by replacing manual and paper-based procedures with a single secure web platform. It reduces the effort needed to file and verify reports, keeps all case information in one place, and gives citizens real-time visibility into the status of their reports.

1.2.2 Primary Goals

[1] Online reporting: allow citizens to submit crime reports with category, description, date, location and optional evidence.

[2] Secure multi-role access: provide separate Citizen, Police Officer and Administrator portals with role-based authorization.

[3] Structured case workflow: support verification, rejection with a reason, investigation updates, resolution and closure of cases.

[4] Transparency: show citizens their report status, investigation remarks and a status timeline, and notify them at each change.

[5] Centralized administration: let administrators manage citizens, police officers and crime categories and view all reports.

[6] Analytics: provide dashboard statistics by status and category, with search and filtering by Case ID, keyword, status, category and date.

[7] Data protection: validate all inputs and uploads, and protect against SQL injection, XSS and CSRF attacks.

1.3 Project Scope

1.3.1 System Boundaries

The project is a Minimum Viable Product (MVP) that was planned for completion within one academic semester. The included features are user registration and login with role-based access control, crime report submission with evidence upload, report tracking and history, help and resources, police verification and investigation management, administrator management of users and categories, in-system notifications, search and filtering, case status history, printable report details, and dashboard analytics.

Several advanced capabilities are deliberately excluded from the current version. These are AI-based crime prediction, face recognition, a dedicated mobile application, GPS tracking and map integration, SMS or email notifications, crime heat maps, live chat support, and court or prison management. They are listed as possible future improvements.

1.3.2 User Categories

Citizens register their own accounts, submit crime reports, upload evidence, track report status, view their report history, receive notifications and access help resources. **Police Officers** log in with accounts created by the administrator, view and search reports, verify or reject them, add investigation updates and close cases. **Administrators** manage citizen and police officer accounts, maintain crime categories, view all reports system-wide and monitor the dashboard statistics.

1.4 Stakeholders

Primary Stakeholders

The primary stakeholders are citizens who report crimes, police officers who verify and investigate reports, and system administrators who manage the platform.

Secondary Stakeholders

The secondary stakeholders are the course supervisors and evaluators (Shad Ahmed and Md Faysal, Lecturers, Department of CSE), the development team Syntax Squad, and any future contributors who may extend the system.

1.5 Project Schedule

The project was planned for a 16-week academic semester covering requirement analysis, design, development, integration, testing and documentation. The table below lists the major phases and the dates recorded in the submitted deliverables.

Phase	Key Deliverables	Documented Date	Status
Phase 1: Planning & Requirements	Software Requirements Specification (SRS) v1.0 with use case diagrams, ER diagram and architecture diagram	06 September 2026	Completed
Phase 2: System Modeling	ER diagram, DFD (Level 0 and 1), use case, class, state and sequence diagrams (UML assignment)	22 October 2026	Completed
Phase 3: Implementation & Integration	Working Citizen, Police Officer and Administrator modules; MySQL database	Completed before testing	Completed
Phase 4: Testing & Defect Fixing	Test Case Report: 136 test cases, 5 defects found and fixed	11 November 2026	Completed
Phase 5: Final Documentation	Project report	14 November 2026	Completed

CHAPTER 2

SOFTWARE REQUIREMENT SPECIFICATION

2.1 Functional Requirements

The functional requirements are grouped by feature. Each requirement is identified by an ID (FR-01 to FR-20). Priority is High for user accounts, report submission, report tracking, police report management and administrator management, and Medium for help and resources.

2.1.1 User Accounts (Priority: High)

ID	Requirement	Actor	Input	Output	Alternative Flow
FR-01	Register	Citizen	Name, email/phone, password	Account created; Login page displayed	Invalid or duplicate email/phone: ask for valid information
FR-02	Login / Logout	Citizen, Police Officer	Registered email/phone and password	Role-specific dashboard shown; after logout, Login page	Wrong credentials: error message, retry
FR-03	Forgot Password	Citizen, Police Officer	Registered email/phone	User can reset password and log in	Unregistered email/phone: error message
FR-04	View / Edit Account Details	Citizen, Police Officer	Profile changes and Save	Account information displayed or updated	Invalid data: ask for valid details

2.1.2 Crime Report Submission (Priority: High)

ID	Requirement	Actor	Input	Output	Alternative Flow
FR-05	Submit Crime Report	Citizen	Category, description, date, location	Report submitted; reference number displayed	Missing required field: ask to complete the form
FR-06	Upload Supporting Evidence	Citizen	One or more image or PDF files	Files uploaded and linked to the report	Unsupported type or too large: ask for a valid file

2.1.3 Report Tracking and History (Priority: High)

ID	Requirement	Actor	Input	Output	Alternative Flow
FR-07	Track Report Status	Citizen	Select Track My Reports and a report	Current status and status history displayed	None
FR-08	View Report History	Citizen	Select Report History	All previous reports and outcomes displayed	No reports: empty history message

2.1.4 Help and Resources (Priority: Medium)

ID	Requirement	Actor	Input	Output	Alternative Flow
FR-09	Access Help and Resources	Citizen	Click Help & Resources	Emergency contacts, safety guidelines and FAQs displayed	None

2.1.5 Police Officer Report Management (Priority: High)

ID	Requirement	Actor	Input	Output	Alternative Flow
FR-10	Police Officer Login	Police Officer	Registered email/phone and password	Police Officer Dashboard displayed	Wrong credentials: error message
FR-11	View Crime Reports	Police Officer	Police Officer Dashboard	Available reports listed	None
FR-12	Verify or Reject a Report	Police Officer	Select Verify or Reject on report details	Status updated to Verified or Rejected; citizen notified	A reason must be provided when rejecting
FR-13	Update Investigation Status	Police Officer	New status and Update	Report status updated and visible to citizen	None
FR-14	Add Investigation Remarks	Police Officer	Remarks and Save	Remarks saved and visible to citizen	None
FR-15	Search Reports	Police Officer	Keyword, category or status filters	Matching reports displayed	No match: "No results found"

2.1.6 Administrator Management (Priority: High)

ID	Requirement	Actor	Input	Output	Alternative Flow
FR-16	Manage Citizen Accounts	Administrator	Add, edit or remove a citizen account	Citizen account list updated	Invalid data: ask for valid details
FR-17	Manage Police Officer Accounts	Administrator	Add, edit or remove an officer account	Police officer list updated	Invalid data: ask for valid details
FR-18	Manage Crime Categories	Administrator	Add, rename or remove a category	Category list available to citizens updated	Duplicate name: choose a different name
FR-19	View All Reports	Administrator	Select View All Reports	All reports and statuses displayed	None
FR-20	View Dashboard Statistics	Administrator	Dashboard loads after login	Totals by status and category displayed	None

2.1.7 Other System Features

[1] Role-based authentication: each user can only access features of their role; unauthorized access shows an access-denied message or redirects to the user's own dashboard.

[2] In-system notifications: citizens are notified when their report status changes.

[3] Search and filter: reports can be searched by keyword, Case ID, status and category, and filtered by date.

[4] Basic dashboard analytics: administrator, police and citizen dashboards show summary counts and charts.

2.2 Non-Functional Requirements

Attribute	Requirement
Performance	Common actions such as login, report submission and status updates should respond within a few seconds under normal conditions. As an academic MVP on a local XAMPP server, large-scale load testing is out of scope.
Safety	User details, reports and evidence files must be protected; regular database backups should be maintained during development; citizens are advised not to include unnecessary highly sensitive information in reports.
Security	Passwords stored in hashed form; role-based access so users cannot reach another role's pages or another user's data; evidence limited to supported image and PDF types within a size limit; sessions and data-changing forms validated (CSRF tokens).
Usability	Simple, user-friendly interface so that citizens can submit reports easily.
Reliability	Reports, user information and status updates must be saved without data loss during normal use.
Scalability	The database should support more users, reports and categories without major changes.
Maintainability	Separate Citizen, Police Officer and Administrator modules make the system easier to maintain and update.
Availability	The system should remain accessible on the local XAMPP server during development and demonstration.
Testability	Each module should be testable separately and together.

2.2 Business Rules

[1] The system has three roles: Citizen, Police Officer and Administrator, each with different permissions.

[2] Citizens can register, submit crime reports, upload evidence and track report history.

[3] Police Officers can view, verify, reject and investigate crime reports.

[4] Administrators can manage users and crime categories and monitor all reports.

[5] Police Officer accounts are created and managed by the Administrator; public registration always creates a Citizen account.

[6] Each crime report must belong to exactly one crime category maintained by the Administrator.

[7] A report can only be marked Resolved after it has been Verified by a Police Officer.

2.4 Operating Environment and Constraints

Item	Description
Client	Modern web browsers on Windows, macOS and Linux
Server	XAMPP (Apache, PHP, MySQL); database crime_reporting_system
Front end	HTML5, CSS3, Bootstrap, JavaScript;
Back end	PHP with PDO and session-based authentication
Development tools	Visual Studio Code; Git and GitHub for version control
Constraints	MVP for one academic semester; evidence limited to images and PDF files; availability and scalability limited to the development and demonstration period
Assumptions	Only citizens submit reports; every report has one category; SMS/email, payment systems and map APIs are not included

2.5 Data Requirements

Entity	Description
Users	Citizens, police officers and administrators, distinguished by a role field
Crime Category	Classification of crimes (for example theft, assault, fraud) maintained by the administrator
Crime Report	A report submitted by a citizen about an incident
Evidence	Image or PDF files attached to a report
Investigations	Investigation updates and remarks recorded by officers
Notifications	In-system messages sent to users about report events
Help & Resource	Emergency contacts, safety guidelines and FAQs
Case Status History	Record of every status change of a report (added in the final implementation)

2.6 Field Layouts and Validation Rules

The SRS defines the following field layouts for the login and report submission forms.

Form	Field	Length	Data Type	Mandatory
Login	Email id / Phone No.	30	Alphanumeric	Yes
Login	Password	16	Alphanumeric	Yes
Report	Category	30	String (dropdown)	Yes
Report	Description	1000	String	Yes
Report	Date of Incident	10	Date	Yes
Report	Location	100	String	Yes
Report	Evidence File	N/A	Image / PDF	No

The implemented application enforces the following validation rules, which were verified through boundary-value testing (Chapter 8).

Input	Rule
Full name	3 to 100 characters
Email	Valid format, at most 100 characters, unique
Password	At least 8 characters; confirmation must match
Phone	At least 6 characters (valid phone format)
Report title	5 to 150 characters
Report description	At least 20 characters
Incident date	Valid date, not in the future
Evidence files	JPG, JPEG, PNG or PDF; maximum 5 MB per file; maximum 5 files per report; file content must match extension
Verification / investigation remarks	At least 5 characters
Category name	3 to 100 characters; must be unique

2.7 Requirement Traceability Matrix

Req ID	Description	Design Reference	Test Reference	Status
FR-01	Citizen registration	Use case, Sec 3.1	IT_13 to IT_26	Implemented
FR-02	Login / Logout	Use case, Sec 3.1	IT_01 to IT_12	Implemented
FR-03	Forgot password	Use case, Sec 3.1	IT_10 (defect D_01)	Removed from scope
FR-04	View / edit account details	Use case, Sec 3.1	Not covered in test report	Not verified
FR-05	Submit crime report	Sequence, Sec 3.5	IT_27, IT_29 to IT_37	Implemented
FR-06	Upload evidence	ER, Sec 3.3	UT_07 to UT_20, IT_28, IT_38 to IT_40	Implemented
FR-07	Track report status	State, Sec 3.6	IT_67, IT_68	Implemented
FR-08	View report history	Class, Sec 3.4	IT_89	Implemented
FR-09	Help and resources	DFD, Sec 3.2	UI screen (Chapter 7)	Implemented
FR-10	Police login	Use case, Sec 3.1	IT_03	Implemented
FR-11	View crime reports	Use case, Sec 3.1	IT_70 to IT_78	Implemented
FR-12	Verify / reject report	Sequence, Sec 3.5	IT_41 to IT_47	Implemented
FR-13	Update investigation status	State, Sec 3.6	IT_48 to IT_56	Implemented
FR-14	Add investigation remarks	ER, Sec 3.3	IT_52, IT_53	Implemented
FR-15	Search reports	Use case, Sec 3.1	IT_70 to IT_80, IT_96	Implemented
FR-16	Manage citizen accounts	Use case, Sec 3.1	IT_23, IT_24 (defect D_02)	Implemented
FR-17	Manage police accounts	Use case, Sec 3.1	IT_95, D_02	Implemented
FR-18	Manage crime categories	ER, Sec 3.3	IT_91 to IT_94	Implemented
FR-19	View all reports	Use case, Sec 3.1	IT_81 to IT_84	Implemented
FR-20	Dashboard statistics	DFD, Sec 3.2	IT_60 to IT_66	Implemented

CHAPTER 3

SYSTEM ANALYSIS & DESIGN

3.1 Use Case Diagram

The use case diagrams show how each actor interacts with the system. The combined diagram gives an overview, and the three role-specific diagrams from the SRS show the detail for each actor.

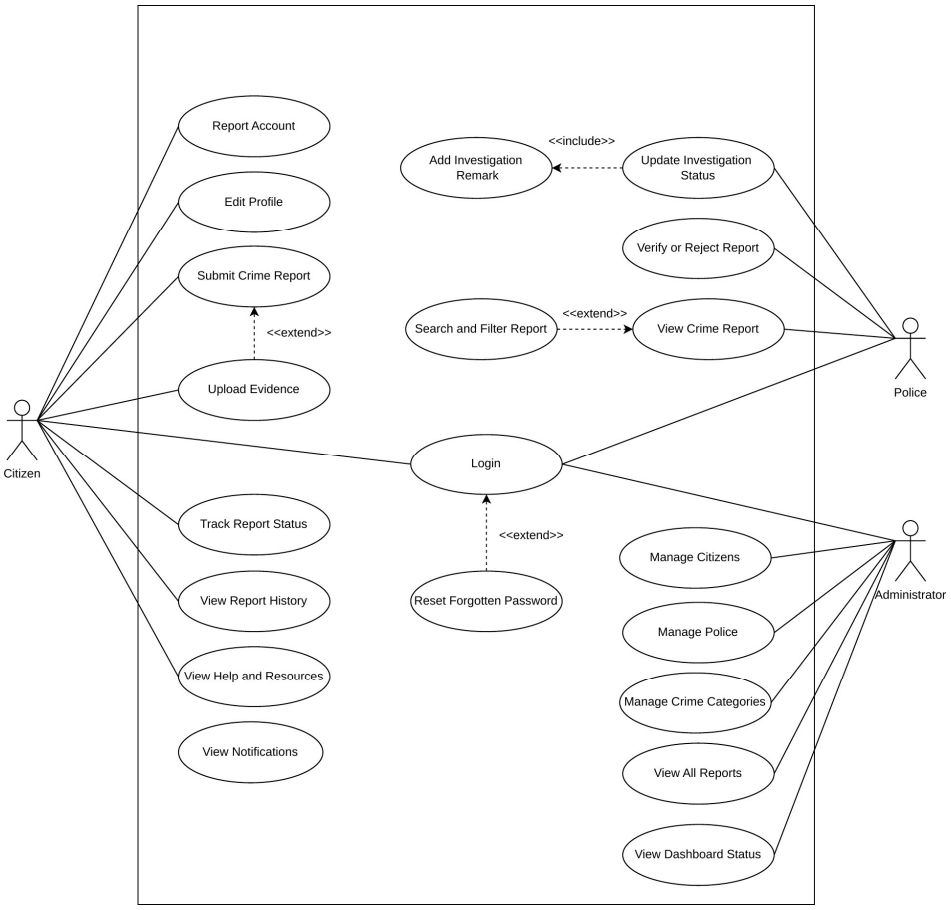


Figure 1: Use Case Diagram of the Online Crime Reporting and Case Management System

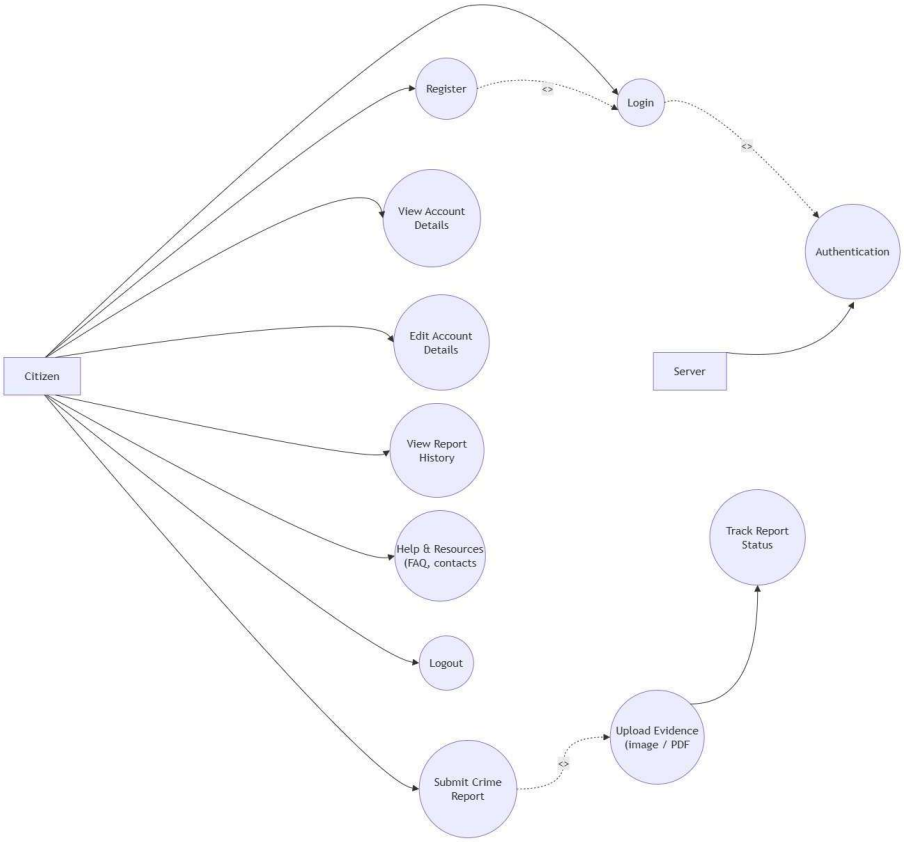


Figure 2: Citizen Use Case Diagram

The Citizen can register, log in, view and edit account details, submit crime reports, view report history, access help and resources and log out. Upload Evidence is an optional extension of Submit Crime Report, because a report can be submitted without evidence.

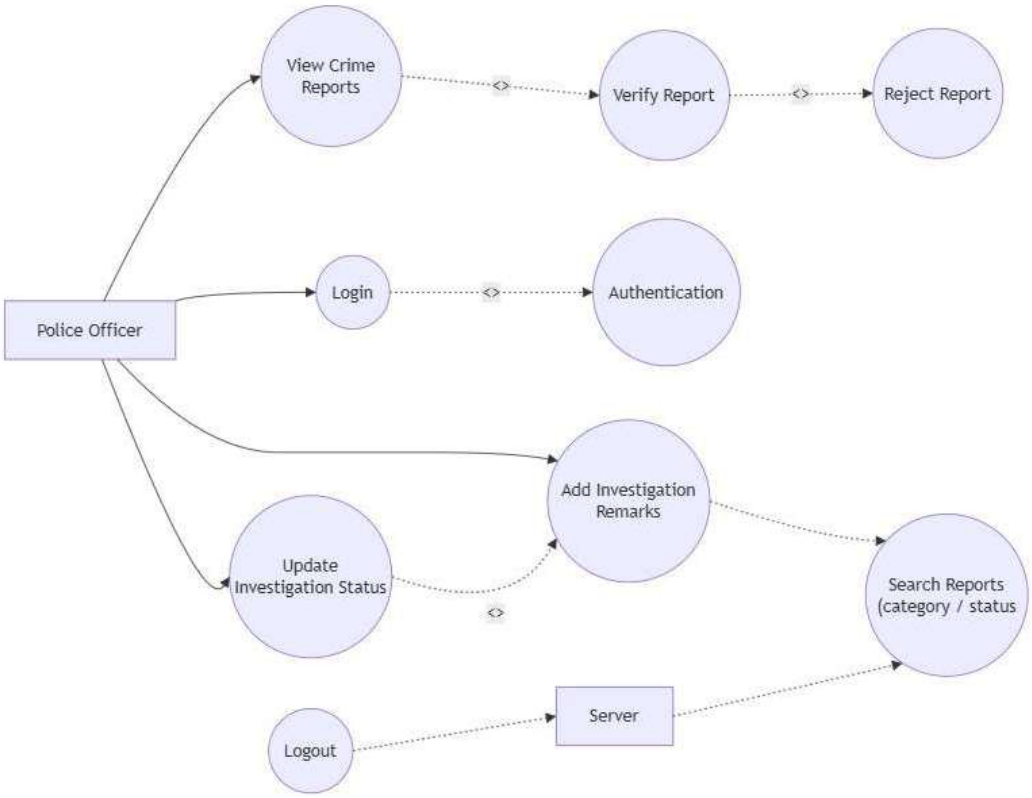


Figure 3: Police Officer Use Case Diagram

The Police Officer can view crime reports, verify a report (with Reject Report as the alternative outcome), update investigation status, add investigation remarks and search reports by category or status.

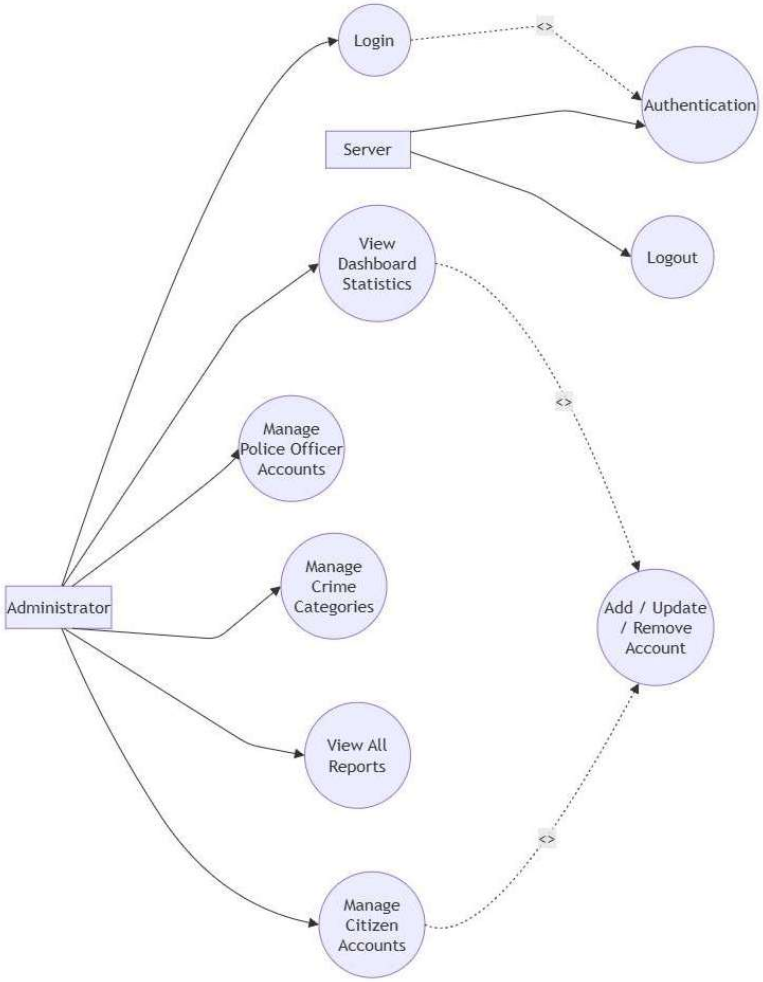


Figure 4: Administrator Use Case Diagram

The Administrator can view dashboard statistics, manage citizen and police officer accounts, manage crime categories and view all reports. Add / Update / Remove Account is a common function used for both account types.

Use Case Descriptions

UC-01: User Login

Attribute	Description
Actor	Citizen, Police Officer, Administrator
Precondition	User account exists in the system
Main Flow	1. User enters email and password. 2. System validates the CSRF token and credentials. 3. System creates a session and redirects to the role-specific dashboard.
Alternative Flow	Empty fields or invalid credentials: an error message is shown and the user can retry.
Postcondition	User is logged in with access limited to their role

UC-02: Submit Crime Report

Attribute	Description
Actor	Citizen
Precondition	Citizen is logged in
Main Flow	1. Citizen selects category and enters title, description, incident date and location. 2. Citizen optionally attaches up to five evidence files. 3. System validates the data and files. 4. System saves the report with status Pending, assigns a Case ID and creates a notification.
Alternative Flow	Validation fails or the file type is invalid: an error or warning is shown; an invalid evidence file is not stored.
Postcondition	Report is stored as Pending and visible to police officers

UC-03: Verify or Reject Report

Attribute	Description
Actor	Police Officer
Precondition	Officer is logged in; report status is Pending
Main Flow	1. Officer opens the report and reviews the details and evidence. 2. Officer selects Verify or Reject and enters remarks. 3. System updates the status, records the verifying officer and time, writes the status history and notifies the citizen.
Alternative Flow	No decision selected or remarks shorter than five characters: an error is shown. A report that is already verified cannot be verified again.
Postcondition	Report status is Verified or Rejected

UC-04: Update Investigation

Attribute	Description
Actor	Police Officer
Precondition	Report has been verified
Main Flow	1. Officer selects Investigating, Resolved or Closed and enters remarks. 2. System inserts an investigation record, updates the report status, writes the status history and notifies the citizen.
Alternative Flow	A Pending report cannot be investigated (verify first); a Closed case is archived and cannot be updated.
Postcondition	Report status and investigation history are updated

UC-05: Manage Crime Categories

Attribute	Description
Actor	Administrator
Precondition	Administrator is logged in
Main Flow	1. Administrator adds, renames or deletes a category. 2. System validates the name (3 to 100 characters, unique). 3. System updates the category list shown in the citizen report form.
Alternative Flow	Duplicate name or invalid length: error shown. A category that already has reports cannot be deleted.
Postcondition	Category list is updated

3.2 Data Flow Diagrams

DFD Level 0

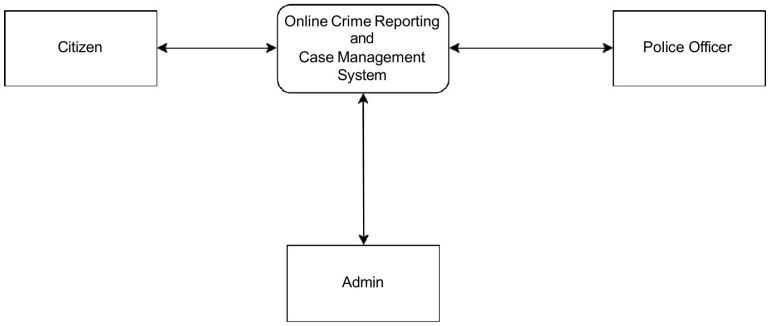


Figure 5: Level 0 DFD (Context Diagram)

The context diagram shows the Online Crime Reporting and Case Management System as a single process that exchanges data with three external entities: Citizen, Police Officer and Admin.

DFD Level 1

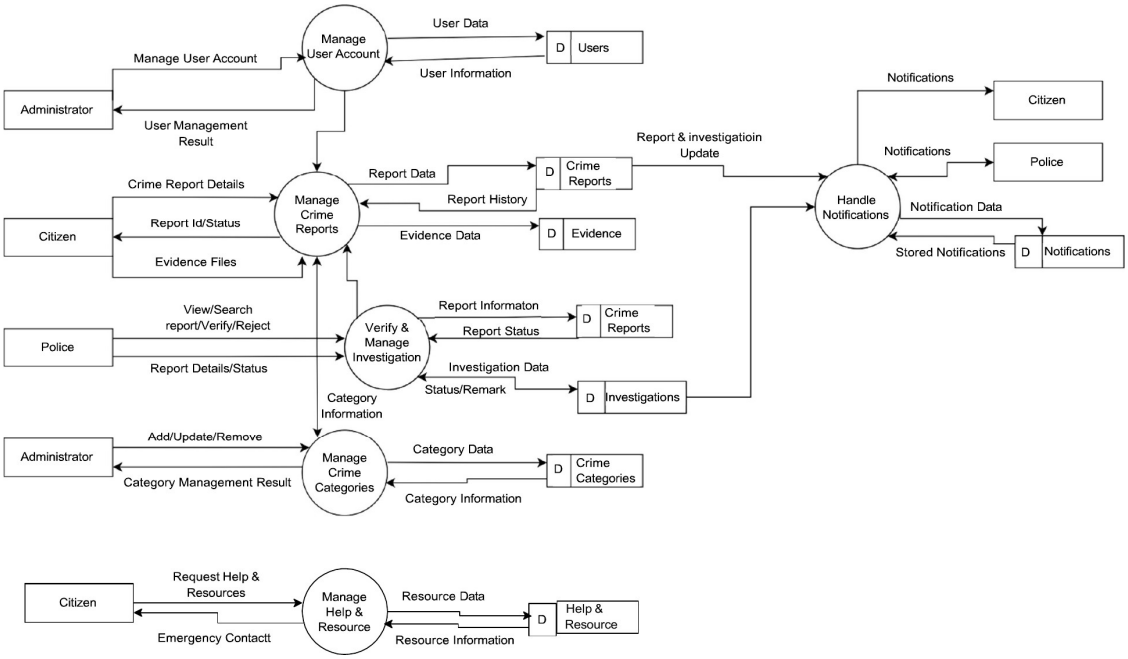


Figure 6: Level 1 DFD

The Level 1 diagram breaks the system into six processes: Manage User Account, Manage Crime Reports, Verify & Manage Investigation, Manage Crime Categories, Manage Help & Resource, and Handle Notifications. They read and write seven data stores: Users, Crime Reports, Evidence, Investigations, Crime Categories, Notifications, and Help & Resource. Report and investigation updates flow into the notification process, which sends notifications to citizens and police.

3.3 Entity Relationship Diagram (ERD)

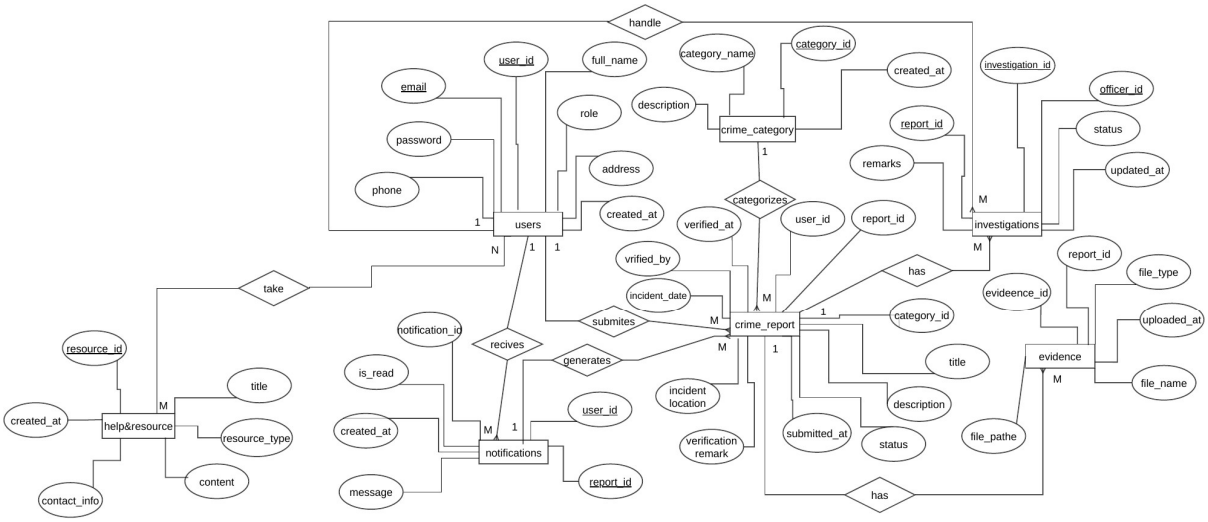


Figure 7: Entity Relationship Diagram

The ERD shows the main entities (users, crime_category, crime_report, evidence, investigations, notifications and help&resource) and their relationships. One citizen can submit many reports; one category classifies many reports; one report can have many evidence files, investigation records and notifications; and one officer can handle many investigations.

Entity Descriptions

Entity: users

Attribute	Key	Description
user_id	PK	Unique identifier of the user
full_name		Full name of the user
email	Unique	Login email address
password		Hashed password
phone		Phone number
address		Address of the user
role		citizen, police or admin
created_at		Account creation time

Entity: crime_category

Attribute	Key	Description
category_id	PK	Unique identifier of the category
category_name	Unique	Name of the category (for example Theft, Fraud)
description		Description of the category
created_at		Creation time

Entity: crime_report

Attribute	Key	Description
report_id	PK	Unique identifier of the report
user_id	FK	Citizen who submitted the report (users)
category_id	FK	Category of the report (crime_category)
title		Short title of the incident
description		Detailed description
incident_date		Date the incident occurred
incident_location		Location of the incident
status		Pending, Verified, Rejected, Investigating, Resolved or Closed
submitted_at		Submission time
verified_by	FK	Officer who verified or rejected the report (users)
verified_at		Time of verification
verification_remark		Remarks recorded at verification

Entity: evidence

Attribute	Key	Description
evidence_id	PK	Unique identifier of the evidence file
report_id	FK	Report the file belongs to (crime_report)
file_name		Stored (unique) file name
file_type		File type (JPG, PNG, PDF)
file_path		Path under uploads/evidence
uploaded_at		Upload time

Entity: Investigations

Attribute	Key	Description
investigation_id	PK	Unique identifier of the investigation update
report_id	FK	Report being investigated (crime_report)
officer_id	FK	Officer who made the update (users)
status		Investigating, Resolved or Closed
remarks		Investigation remarks
updated_at		Time of the update

Entity: notifications

Attribute	Key	Description
notification_id	PK	Unique identifier of the notification
user_id	FK	Recipient (users)
report_id	FK	Related report (crime_report)
message		Notification text
is_read		Whether the user has read it
created_at		Creation time

Entity: Help & resource

Attribute	Key	Description
resource_id	PK	Unique identifier of the resource
title		Title of the resource
resource_type		Emergency contact, safety guideline or FAQ
content		Resource text
contact_info		Contact number where applicable
created_at		Creation time

3.4 Class Diagram

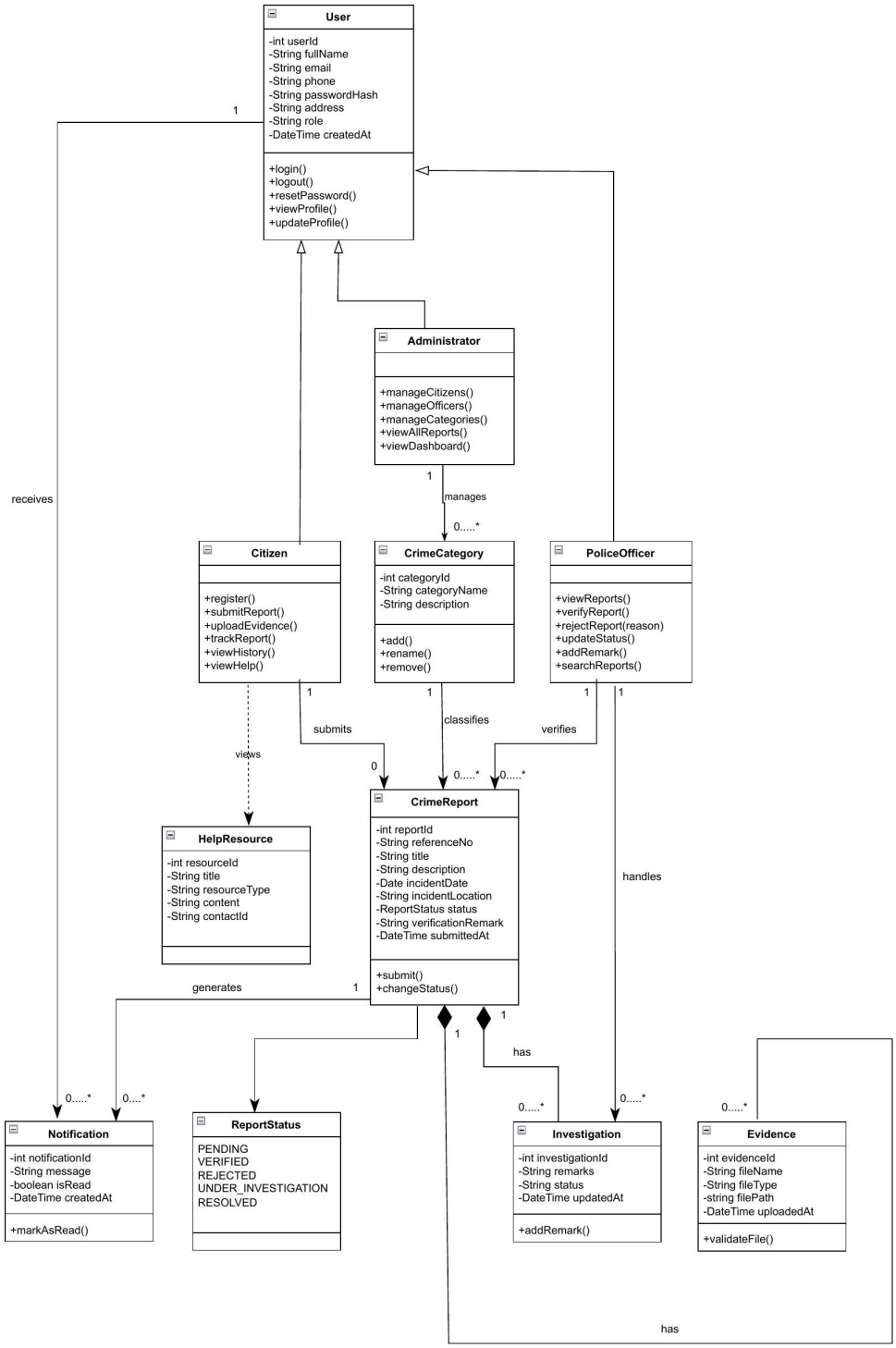


Figure 8: Class Diagram

The class diagram shows the object-oriented design. **User** holds the common attributes (userId, fullName, email, phone, passwordHash, address, role, createdAt) and operations (login, logout, resetPassword, viewProfile, updateProfile). **Citizen**, **PoliceOfficer** and **Administrator** specialize User. The remaining classes are **CrimeReport**, **CrimeCategory**, **Evidence**, **Investigation**, **Notification**, **HelpResource** and the **ReportStatus** enumeration (PENDING, VERIFIED, REJECTED, UNDER_INVESTIGATION, RESOLVED).

A citizen submits zero or many reports; each report belongs to one category, can have many evidence files, can be handled through many investigation updates, and generates notifications for the citizen. The final implementation adds the Closed status after Resolved.

3.5 Sequence Diagram

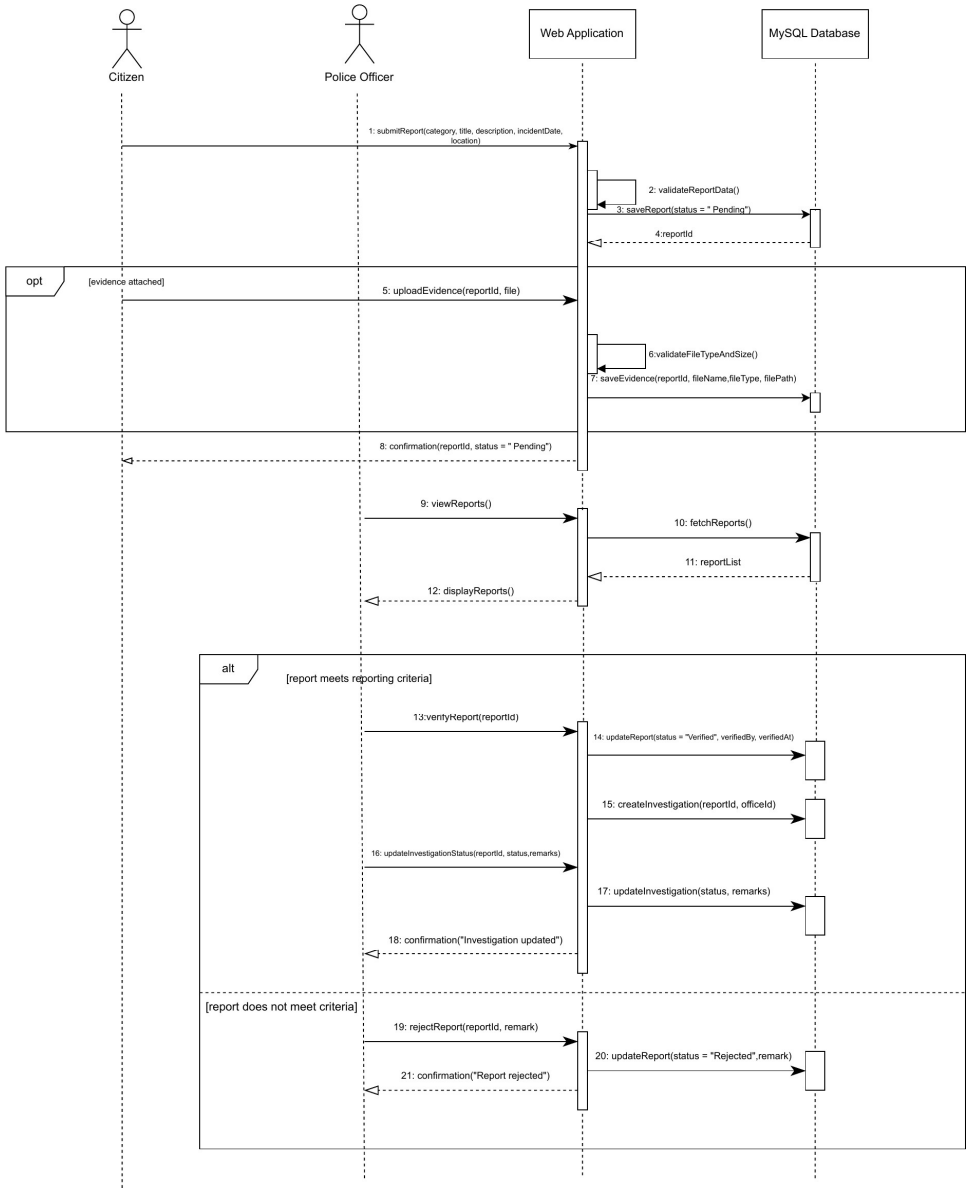


Figure 9: Sequence Diagram: Report Submission, Verification and Investigation Update

The sequence diagram traces the full flow between the Citizen, Police Officer, Web Application and MySQL Database. The citizen submits the report, the application validates it and saves it as Pending, and optional evidence is validated and stored. The officer then views the reports and either verifies the report (creating an investigation and updating its status and remarks) or rejects it with a remark.

3.6 State Diagram

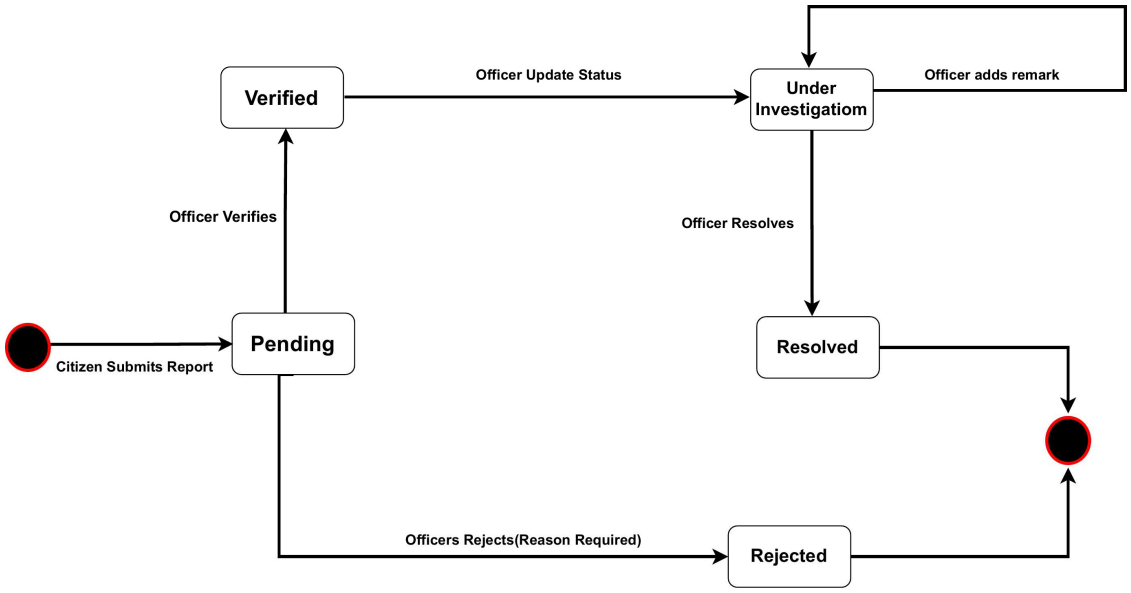


Figure 10: State Diagram of a Crime Report

A report starts in the Pending state when the citizen submits it. The officer either verifies it or rejects it (a reason is required). A verified report moves to Under Investigation when the officer updates its status, and then to Resolved. In the final implementation a resolved case can be formally Closed.

From	To	Trigger
(start)	Pending	Citizen submits report
Pending	Verified	Officer verifies the report
Pending	Rejected	Officer rejects the report (reason required)
Verified	Investigating	Officer adds an investigation update
Investigating	Resolved	Officer marks the case as resolved
Resolved	Closed	Officer finalizes and closes the case file

CHAPTER 4
DATABASE DESIGN

4.1 Database Schema Overview

The system uses a relational database in MySQL named crime_reporting_system, created from the database.sql script. It consists of eight tables that cover users, categories, reports, evidence, investigations, notifications, help resources and case status history. The script also inserts the seed data: one administrator account (admin@crms.local), eight crime categories and six help resources.

4.2 Table Descriptions

No.	Table	Purpose
1	users	Stores citizen, police officer and administrator accounts with their role and hashed password
2	crime_category	Stores the crime categories managed by the administrator
3	crime_report	Stores submitted reports with status, Case ID source data, verification details and submission time
4	evidence	Stores metadata of uploaded evidence files linked to a report
5	investigations	Stores each investigation update with its officer, status, remarks and time
6	notifications	Stores in-system notifications with a read/unread flag
7	help_resource	Stores emergency contacts, safety guidelines and FAQs
8	case_status_history	Stores every status change of a report with the previous status, new status, remarks and officer id

The Case ID (format CR-YYYY-NNNNN) is derived from the report id and the year of submission. Table names above follow the ER diagram; the implemented script uses plural names such as crime_reports.

4.3 Database Relationships

From Table	To Table	Relationship	Key / Rule
crime_report	users (citizen)	Many to One	Foreign key user_id
crime_report	crime_category	Many to One	Foreign key category_id
crime_report	users (officer)	Many to One	verified_by
evidence	crime_report	Many to One	Foreign key report_id
investigations	crime_report	Many to One	Foreign key report_id
investigations	users (officer)	Many to One	Foreign key officer_id
notifications	users	Many to One	Foreign key user_id
notifications	crime_report	Many to One	Foreign key report_id
case_status_history	crime_report	Many to One	Foreign key, ON DELETE CASCADE
case_status_history	users	Many to One	Foreign key, ON DELETE SET NULL

Integrity rules enforced by the application: a category that has reports cannot be deleted, and an officer who has investigation records cannot be deleted.

CHAPTER 5

SYSTEM ARCHITECTURE & TECHNOLOGY STACK

5.1 Architecture Overview

CRMS follows a three-tier client-server architecture:

3 **Presentation Layer:** web interface for Citizens, Police Officers and Administrators (HTML5, CSS3, Bootstrap, JavaScript).

4 **Application Layer:** PHP application logic running on the Apache web server.

5 **Data Layer:** MySQL database storing users, reports, evidence, investigations and case records.

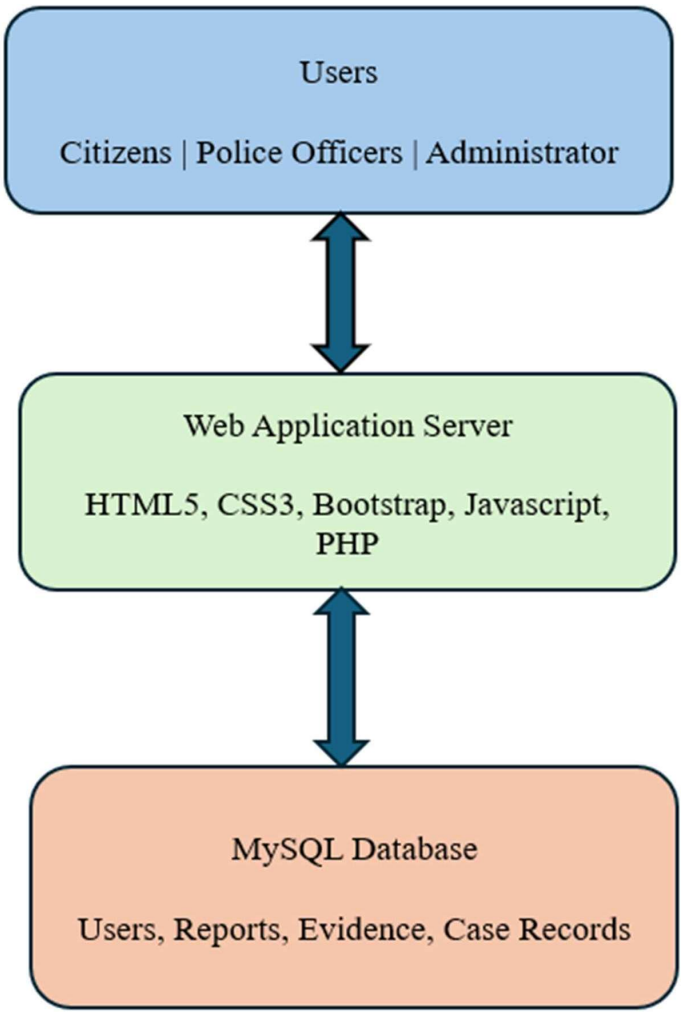


Figure 11: System Architecture Diagram

5.2 Technology Stack

Backend Technologies

Component	Technology
Server-side language	PHP (PDO for database access)
Database	MySQL (port 3306, configured in config/database.php)
Web server	Apache (XAMPP)
Session management	PHP sessions with role-based access control

Frontend Technologies

Component	Technology	Purpose
Markup	HTML5	Page structure
Styling	CSS3, Bootstrap	Visual design and layout
Scripting	JavaScript	Client-side behavior
Charts	Chart.js (CDN)	Status and category charts; internet needed to display charts

5.3 Development and Testing Tools

Tool	Purpose
XAMPP	Local development and test server
Visual Studio Code	Code editor
Git and GitHub	Version control
PHPUnit	Unit testing of helper functions
Web browser	Functional and integration testing

5.4 Security Architecture

6 **Authentication and authorization:** session-based login; require_login and require_role guard every page.

7 **CSRF protection:** a 64-character hexadecimal token is generated per session and verified on every data-changing form.

8 **SQL injection prevention:** all queries use PDO prepared statements.

9 **XSS prevention:** output is escaped with htmlspecialchars through sanitize_input and output encoding.

10 **Secure uploads:** extension, size, MIME content and file-count checks, plus randomized file names to prevent path traversal.

11 **Ownership checks:** a citizen can only open their own reports.

12 **Role integrity:** the role is fixed to citizen on public registration, so role escalation through the request is blocked.

CHAPTER 6

IMPLEMENTATION

6.1 System Modules

The system consists of the following modules, each serving a specific role in the case-handling process.

1. Authentication and Access Control Module

Handles registration, login, logout and sessions. The functions is_logged_in, login_user, logout_user, require_login, require_role and redirect_to_dashboard implement role-based access, redirecting each user to the correct dashboard and blocking unauthorized pages with an appropriate message.

2. Citizen Module

Allows citizens to register, view their dashboard summary, submit reports with evidence, track report status through a timeline, view report history, read notifications and open help and resources.

3. Police Officer Module

Provides the officer portal: dashboard with case lifecycle distribution, pending verification queue, report search and filtering, the Verify or Reject page, the investigation page (Investigating, Resolved, Closed) and the report details page.

4. Administrator Module

Provides the executive dashboard, management of citizens, police officers and crime categories, and a system-wide reports page with filtering and a full report details page.

5. Report and Evidence Management Module

Validates and stores reports, generates Case IDs, validates evidence files (up to five files, 5 MB each, JPG, JPEG, PNG or PDF) and stores them under uploads/evidence with unique names.

6. Investigation and Case Lifecycle Module

Implements the workflow Pending, Verified, Investigating, Resolved and Closed, and Rejected at verification. Each change inserts an investigation record, updates the report status and notifies the citizen.

7. Notification Module

Creates an in-system notification whenever a report is submitted, verified, rejected or updated, with a New indicator until it is read.

8. Case Status History Module

Records each status change (previous status, new status, remarks and officer) in the case_status_history table so that the complete life of a case can be audited.

9. Dashboard, Search and Reporting Module

Provides the citizen, police and administrator dashboards (including Chart.js doughnut and bar charts), search by Case ID, keyword, status and category, date filters (last 7 days, last 30 days, this month, exact date), and a Print / Export PDF option on report details pages.

6.2 Core Helper Functions

Function	Purpose
sanitize_input()	Trims input and escapes HTML special characters
validate_file_upload()	Checks upload errors, size limit, extension and real file content
generate_unique_filename()	Creates a random, collision-free file name keeping only the extension
get_status_badge_class()	Maps a case status to a display badge style
generate_csrf_token() / verify_csrf_token()	Creates and verifies the per-session CSRF token
format_date()	Formats a date-time for display
format_case_id()	Builds the Case ID (CR-YYYY-NNNNN) from the report id and submission year
redirect(), set_flash_message(), get_flash_message()	Page redirection and one-time user messages
create_notification()	Creates an in-system notification
create_case_status_history() / get_case_status_history()	Writes and reads the case status history

6.3 Key Features Implemented

Feature	Description	Status
Multi-role authentication	Separate portals for Citizen, Police Officer and Administrator	Complete
Crime report submission	Validated form with category, description, date and location	Complete
Evidence upload	Secure upload of up to 5 image/PDF files	Complete
Case lifecycle	Pending, Verified, Investigating, Resolved, Closed, Rejected	Complete
Case ID	Unique CR-YYYY-NNNNN identifier for each report	Complete
Report tracking	Status timeline and history for citizens	Complete
Notifications	In-system notification at each status change	Complete
Case status history	Audit trail of status changes	Complete
Search and filters	Case ID, keyword, status, category and date filters	Complete
Dashboards and charts	Citizen, police and admin dashboards with Chart.js charts	Complete

Feature	Description	Status
User and category management	Admin management of citizens, officers and categories	Complete
Help & Resources	Emergency contacts, safety guidelines and FAQs	Complete
Print / Export PDF	Print view of report details pages	Complete

6.4 Differences Between the SRS and the Final Implementation

13 A **Closed** status was added after Resolved so that officers can formally archive a case; the SRS listed Pending, Verified, Rejected, Under Investigation and Resolved.

14 **Case IDs**, case status history, advanced date filters and a printable details view were added during implementation.

15 The **Forgot Password** feature (FR-03) was removed from the login page after testing found a broken link (defect D_01); password reset is outside the approved scope and the reset_token columns remain available for a later feature.

CHAPTER 7

USER INTERFACE

7.1 Interface Design Principles

The interface is designed for simplicity and clarity. Consistency is kept through a common card-based layout, a top navigation bar, and colored status badges that show the case status at a glance. Each role has its own dashboard with summary cards, so users see only the information relevant to them. Forms show clear validation messages, and one-time flash messages confirm each action.

7.2 Main User Interface Screens

Login and Registration

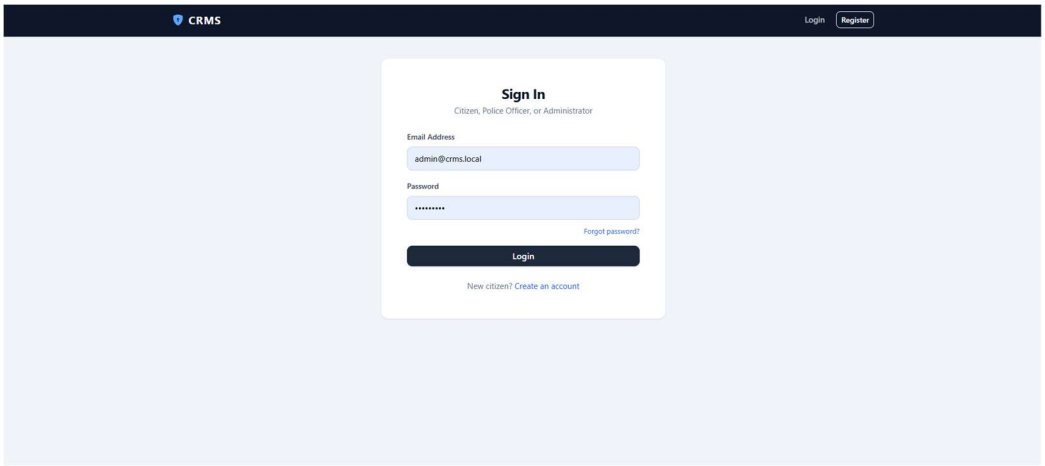


Figure 12: Login Screen

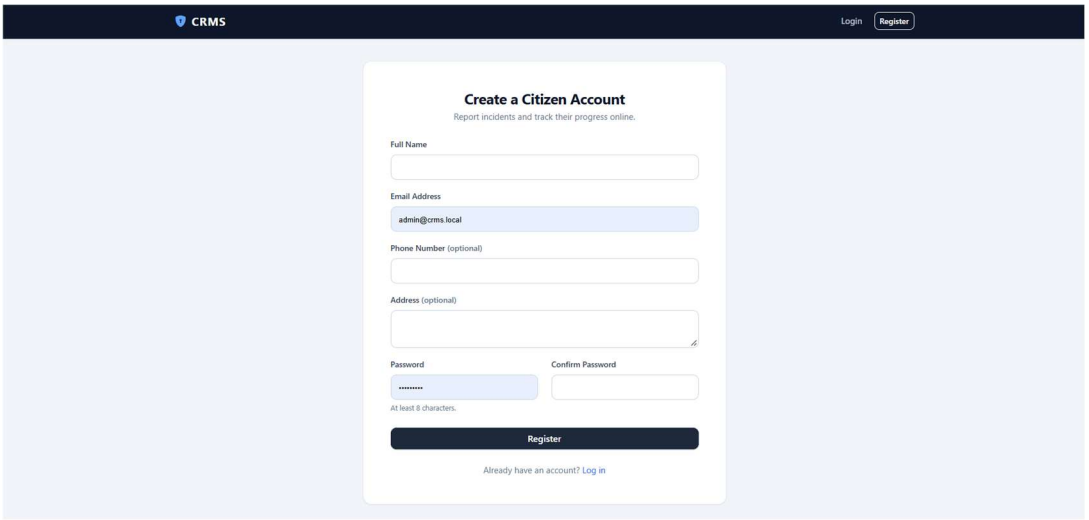


Figure 13: Citizen Registration Screen

The Sign In page serves citizens, police officers and administrators with one form: email, password and a CSRF-protected Login button, with a link to create a citizen account. The

registration page collects full name, email, optional phone and address, and a password with confirmation, and always creates a citizen account.

Citizen Portal

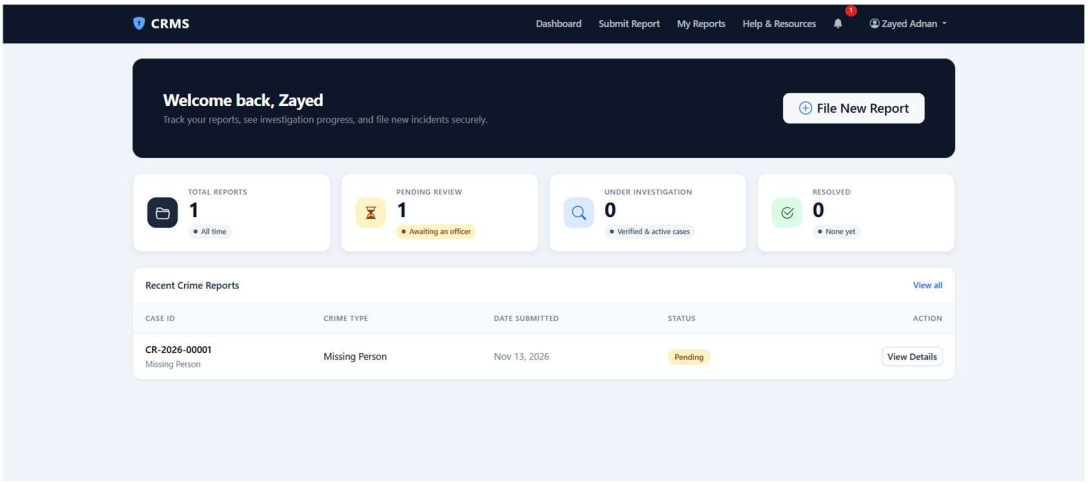


Figure 14: Citizen Dashboard (new report pending)

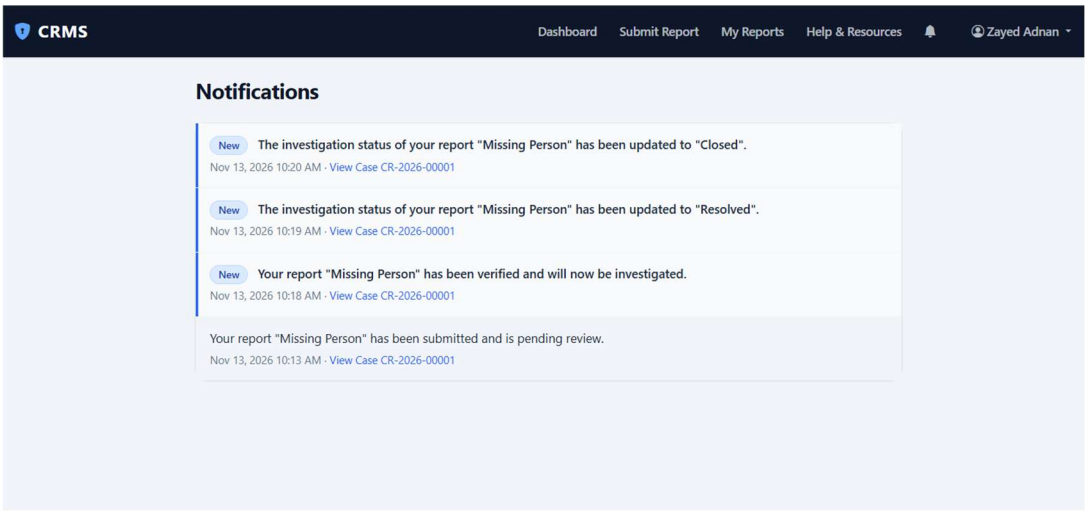


Figure 15: Citizen Notifications

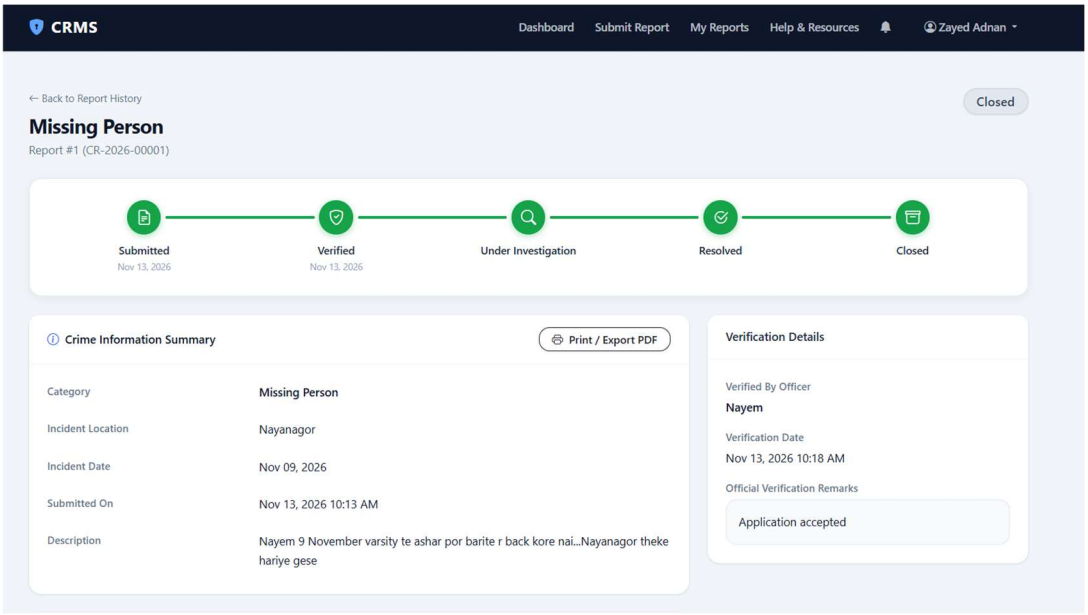


Figure 16: Report Tracking with Status Timeline (closed case)

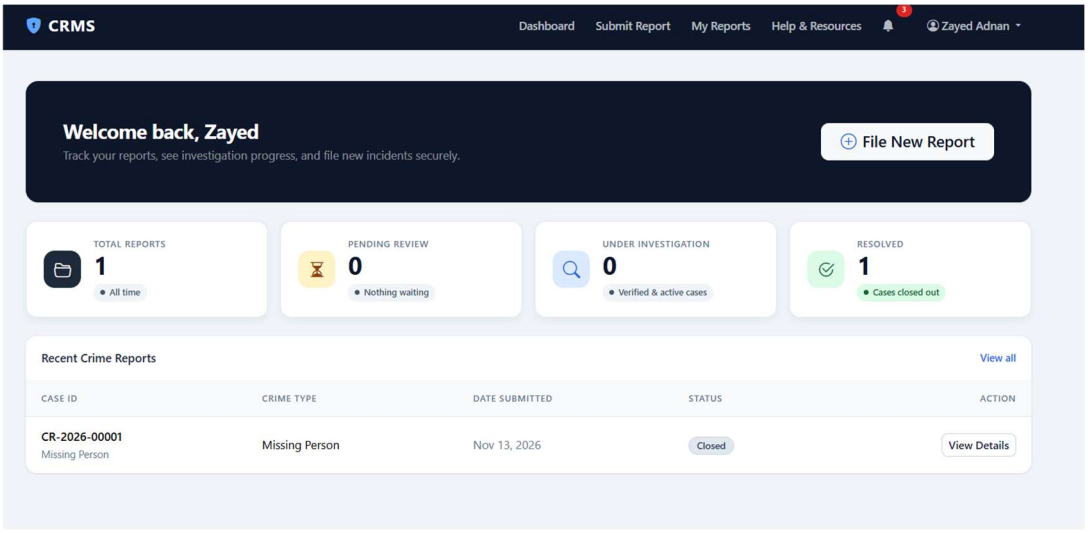


Figure 17: Citizen Dashboard after the Report is Closed

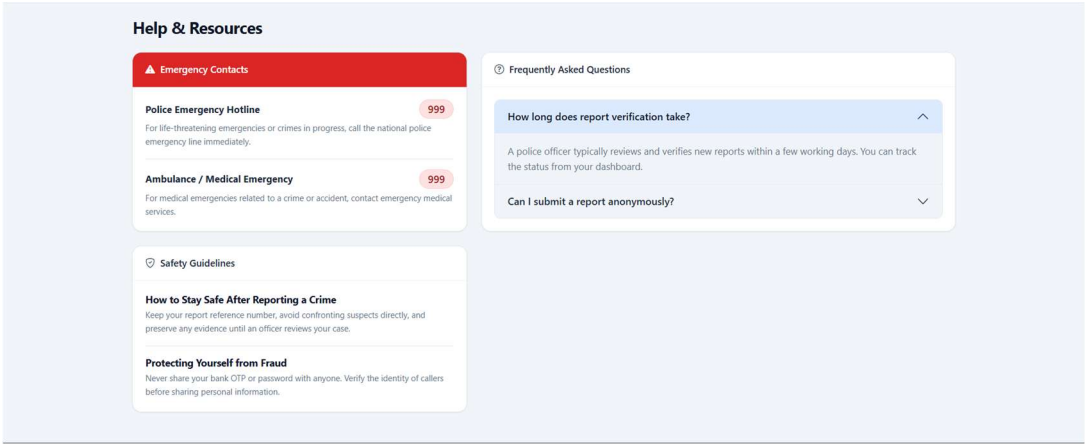


Figure 18: Help & Resources Page

The citizen dashboard shows total, pending, under-investigation and resolved counts with a recent reports table and a File New Report button. The notifications page lists each event with

a New badge and a link to the case. The tracking page shows the five-step timeline (Submitted, Verified, Under Investigation, Resolved, Closed), the crime information summary and the verification details. The help page lists emergency contacts, safety guidelines and FAQs.

Police Officer Portal

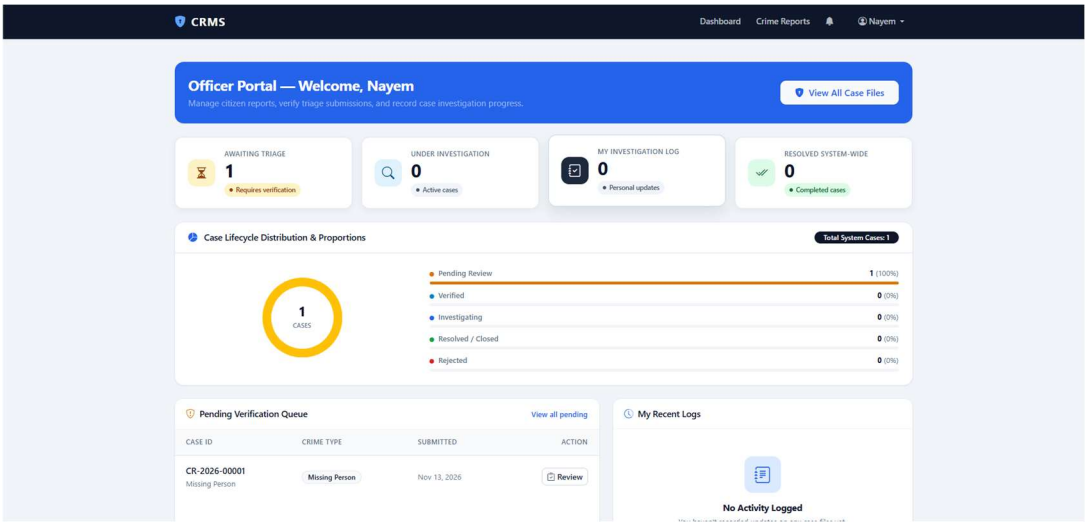


Figure 19: Police Officer Dashboard

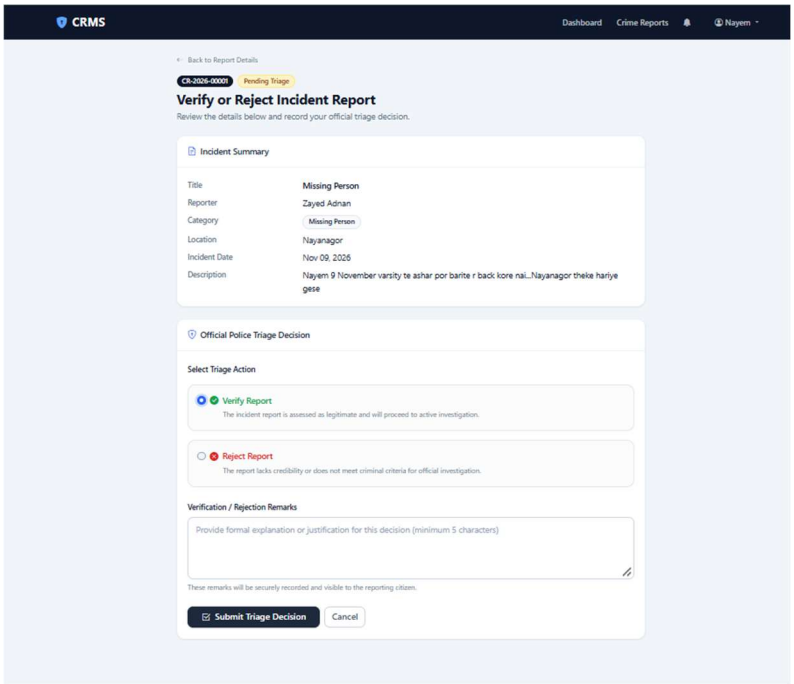


Figure 20: Verify or Reject Incident Report

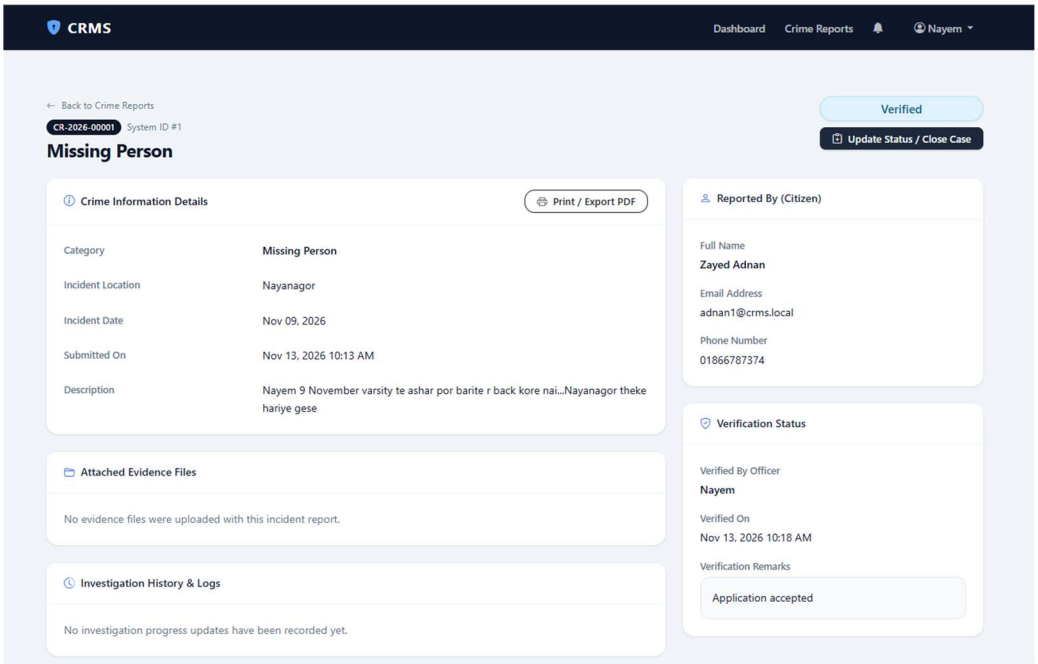


Figure 21: Report Details after Verification

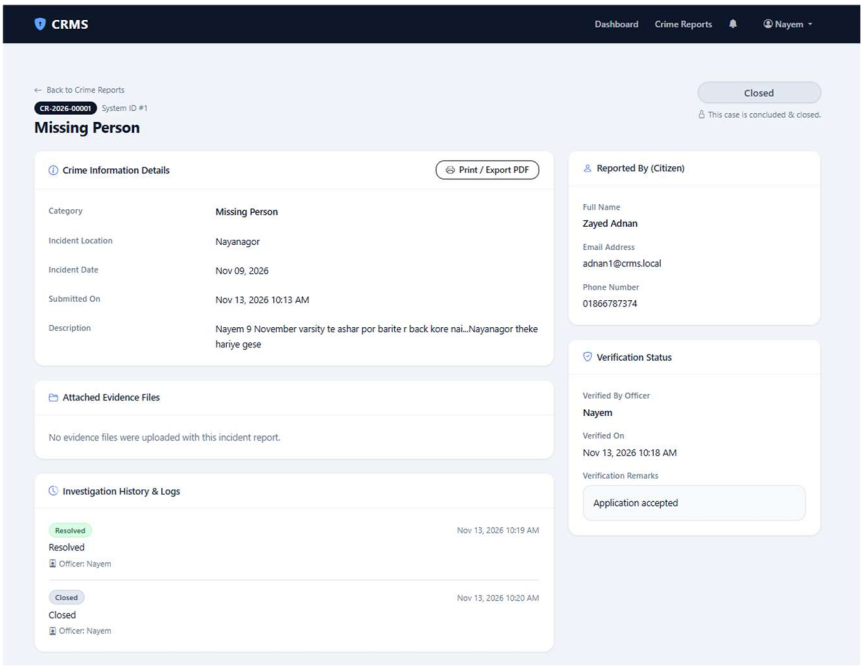


Figure 22: Closed Case with Investigation History

The officer dashboard shows awaiting-triage, under-investigation, personal-update and resolved counts, a lifecycle distribution chart and the pending verification queue. On the Verify or Reject page the officer reviews the incident summary, selects a decision and enters remarks of at least five characters. After verification, the details page shows evidence, investigation history and an Update Status / Close Case button; a closed case shows its full investigation history and is archived.

Administrator Portal

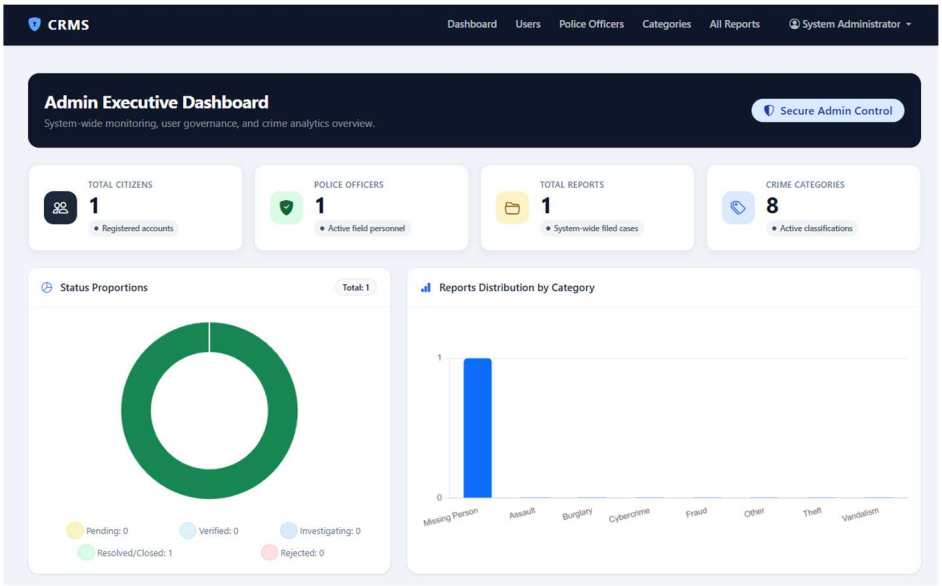


Figure 23: Administrator Executive Dashboard

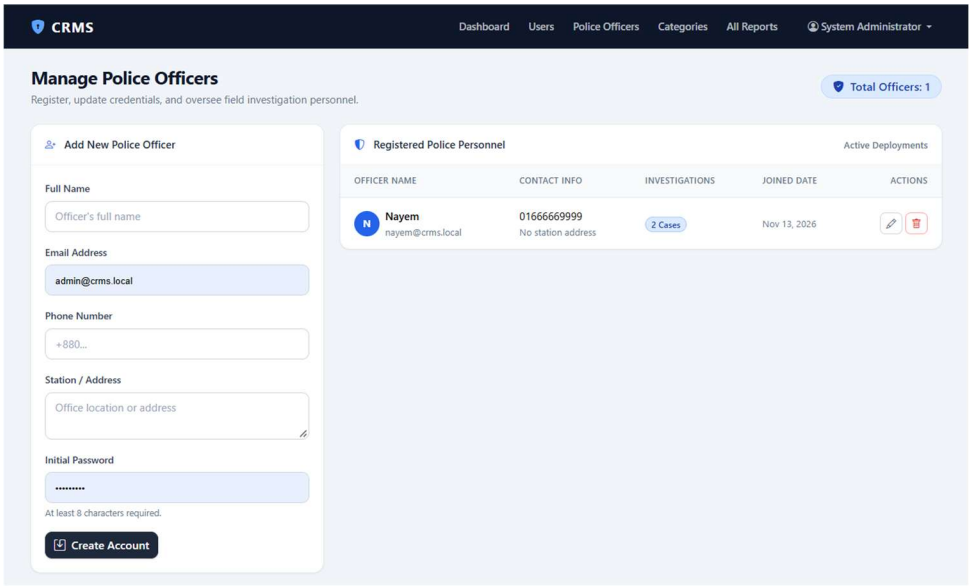


Figure 24: Manage Police Officers

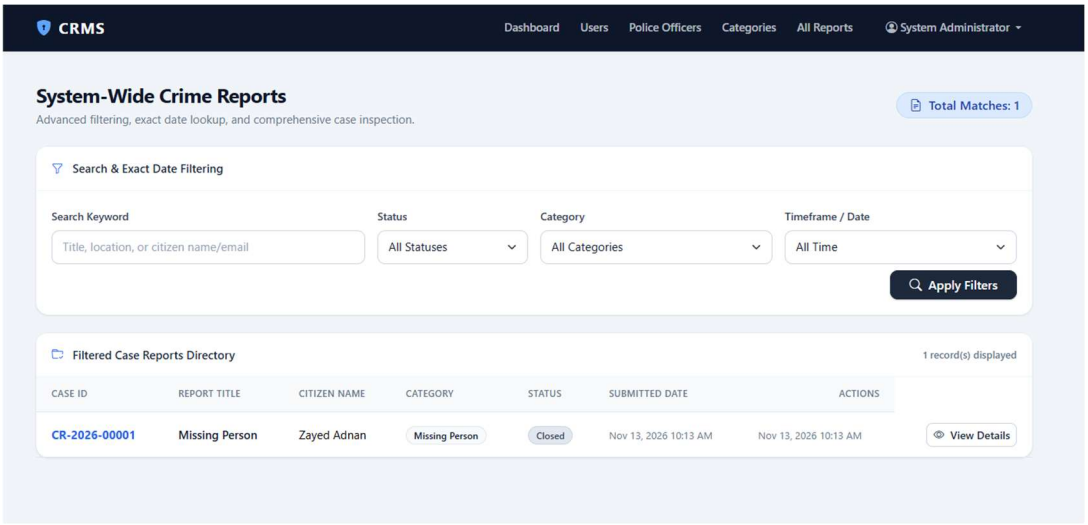


Figure 25: System-Wide Crime Reports

The administrator dashboard shows total citizens, police officers, reports and categories, with a status doughnut chart and a category bar chart. The Manage Police Officers page creates officer accounts and lists registered personnel with their investigation counts. The System-Wide Crime Reports page provides keyword, status, category and date filtering over all case files.

CHAPTER 8

SOFTWARE TESTING

8.1 Testing Objective and Environment

The objective was to design and execute a testing strategy covering unit tests of the core helper functions and integration tests of the key end-to-end flows for all three roles, and to record the defects found and fixed. Testing used XAMPP (Apache, PHP, MySQL on port 3306) with the database created from database.sql, PHPUnit for unit tests and browser-based functional testing. The seed data comprised the administrator admin@crms.local, 8 crime categories and 6 help resources, together with two citizen accounts (for ownership checks) and one police officer account.

8.2 Test Strategy

16 **Equivalence partitioning:** valid and invalid input classes, for example allowed and disallowed file types and valid and invalid email formats.

17 **Boundary value analysis:** values at and just beyond each limit (name length 2 and 3, password 7 and 8, title 4, 5, 150 and 151, description 19 and 20, file size 5,242,880 and 5,242,881 bytes, 5 and 6 evidence files, incident date today and tomorrow).

18 **Negative and security testing:** SQL injection, script tags, invalid CSRF token, role escalation, disguised file uploads and access to another user's data.

19 **Integration flows:** register, login, submit report, verify report, investigate, resolve, close and notifications at each step. Unit tests follow the Arrange, Act, Assert pattern.

8.3 Test Execution Summary

Unit tests

Function	Total	Pass	Fail
sanitize_input()	6	5	1
validate_file_upload()	14	14	0
generate_unique_filename()	3	3	0
get_status_badge_class()	4	4	0
CSRF token functions	5	5	0
format_date()	2	2	0
is_logged_in()	3	3	0
format_case_id()	3	3	0
Total	**40**	**39**	**1**

Integration tests

Module	Total	Pass	Fail
Login	12	11	1
Registration	14	12	2
Report Submission	14	14	0
Police Verification	7	7	0
Investigation	9	9	0
Case Status History	3	1	2
Dashboards and Case Tracking	13	12	1
Filters and Report Details	13	13	0
Access Control	5	5	0
Admin Management	6	6	0
Total	**96**	**90**	**6**

Overall results

Suite	Total	Pass	Fail
Unit tests	40	39	1
Integration tests	96	90	6
All tests (before fixes)	**136**	**129**	**7**

The pass rate before fixes was 94.9%. After the fixes in Section 8.5 were applied, all 7 failed cases were re-run and passed, giving 136 of 136 passed.

8.4 Coverage Summary

Area	Covered by	Coverage
Helper functions (functions.php): 13 functions	UT_01 to UT_34 and integration tests; only get_case_status_history is never called from a page	12 of 13 (92%)
Case ID helper (format_case_id)	UT_38 to UT_40, IT_69	100%
Authentication functions (auth.php)	UT_35 to UT_37, IT_01 to IT_12, IT_86 to IT_90	100%
Citizen features	IT_13 to IT_40, IT_67, IT_68	Core flows covered
Police features	IT_41 to IT_59, IT_70 to IT_72	Core flows covered
Dashboards	IT_60 to IT_72	All three dashboards
Search, filters and details pages	IT_70 to IT_85	All filters and details page
Admin features	IT_91 to IT_96	Category and officer rules
Security controls	UT_03, UT_15, UT_30 to UT_32, IT_08, IT_09, IT_25, IT_26, IT_86 to IT_90	Covered

8.5 Defects Found and Fixed

ID	Description	Found By	Severity	Fix Applied	Retest
D_01	Broken link: "Forgot password?" on the login page points to forgot_password.php, which does not exist (HTTP 404).	IT_10	Medium	Removed the link from the login form; password reset is outside the approved scope.	Pass
D_02	Registration crashes on a name or email longer than 100 characters (unhandled "Data too long" database error, blank page).	IT_23, IT_24	Medium	Added validation: name 3 to 100 characters and email at most 100 characters in register.php, admin/police.php and admin/users.php.	Pass
D_03	Case status history never recorded: create_case_status_history() was never called.	IT_57, IT_58	Medium	Call added after verify/reject and after each investigation update.	Pass
D_04	Police dashboard shows different totals: the Resolved card ignored Closed cases while the lifecycle breakdown counted both.	IT_63	Low	Card now uses the same Resolved-or-Closed total.	Pass
D_05	sanitize_input() removed backslashes from valid text because of stripslashes().	UT_05	Low	Removed stripslashes(); trim() and htmlspecialchars() are kept.	Pass

8.6 Selected Test Cases

The table lists representative cases from each module, including all seven cases that initially failed. The complete set of 136 cases is in the Test Case Report.

ID	Scenario	Expected Result	Actual Result	Status
UT_05	Backslash in input (C:\Users\evidence)	Backslashes preserved	Backslashes removed by stripslashes	Fail
UT_12	File size 5,242,881 bytes	Rejected: size must not exceed 5 MB	Rejected with the same message	Pass
UT_15	PHP script disguised as shell.jpg	Rejected: content does not match extension	Rejected	Pass
IT_01	Valid admin login	Admin dashboard with welcome message	As expected	Pass
IT_08	SQL injection in login email	Access denied, no database error	Generic error, no access	Pass
IT_09	Invalid CSRF token	Form blocked with error	Login blocked	Pass
IT_10	Forgot password link	Password reset page opens	HTTP 404 Not Found	Fail
IT_23	Name of 101 characters	Validation error	Unhandled database error, blank page	Fail
IT_24	Email of 101 characters	Validation error	Unhandled database error, blank page	Fail
IT_26	Role escalation (role=admin in POST)	Account created as citizen	Created as citizen	Pass
IT_30	Title of 5 characters (boundary)	Report accepted	Accepted	Pass
IT_39	Six evidence files	Error: up to 5 files allowed	Error shown, nothing saved	Pass

ID	Scenario	Expected Result	Actual Result	Status
IT_41	Verify a pending report	Status Verified; citizen notified	As expected	Pass
IT_50	Close a resolved case	Status Closed; citizen notified	As expected	Pass
IT_57	History recorded on verification	One history row	No row found	Fail
IT_58	History recorded on investigation update	One history row	No row found	Fail
IT_63	Police dashboard Resolved card	Shows 2 (Resolved + Closed)	Showed 1	Fail
IT_89	Citizen opens another citizen's report	Not found / not authorized	Message shown, redirected	Pass
IT_93	Delete a category that has reports	Deletion blocked	Blocked with explanation	Pass

8.7 Defect Categories Not Covered by the Current Tests

20 **Brute-force protection:** login has no attempt limit or lockout.

21 **Session handling:** automatic session timeout or idle logout was not tested.

22 **Status transition rules:** an officer can move a Resolved case back to Investigating; no rule is specified, so no test asserts it.

23 **Multi-step consistency:** the status update and the investigation row are saved as separate queries without a transaction.

24 **Concurrency:** two officers acting on the same report at the same time were not load-tested.

25 **Offline use:** Chart.js is loaded from a CDN, so charts were not tested without internet.

26 **Performance:** behavior with large numbers of reports, search results and notifications.

27 **Upload security:** no antivirus or image re-encoding test.

28 **Usability and compatibility:** responsive layout and different browsers were not covered.

CHAPTER 9
CONCLUSION & FUTURE WORK

9.1 Project Summary

The Online Crime Reporting and Case Management System has been developed and tested as a web-based solution for reporting and managing crimes. It meets its primary objectives: citizens can report crimes with evidence and track them, police officers can verify and investigate cases through a defined lifecycle, and administrators can manage users and categories and monitor the system.

9.2 Key Accomplishments

29 Complete SRS with 20 functional requirements, non-functional requirements and a traceability matrix.

30 Full set of analysis and design models: use case, DFD (Level 0 and 1), ER, class, state and sequence diagrams.

31 Working Citizen, Police Officer and Administrator modules on a MySQL database with eight tables.

32 Case lifecycle with Case IDs, notifications and a complete case status history.

33 Strong security controls: CSRF protection, prepared statements, output escaping, role and ownership checks and validated uploads.

34 A suite of 136 test cases designed with equivalence partitioning and boundary value analysis; all 7 initial failures (5 defects) were fixed and re-tested to a 100% pass.

9.3 System Benefits

35 **Less manual work:** online submission and verification replace paper-based reporting.

36 **Transparency:** citizens see status, remarks and timeline for each report and receive notifications.

37 **Better organization:** all reports, evidence and investigation records are in one database with search and filters.

38 **Accountability:** the status history records who changed a case and when.

39 **Informed decisions:** dashboards give a quick view of cases by status and category.

9.4 Future Enhancements

The following improvements are planned for future versions, based on the features excluded from the MVP and the gaps found in testing:

40 SMS and email notifications and a working password-reset feature.

41 Login attempt limiting and session timeout.

42 Rules for allowed status transitions and database transactions for multi-step updates.

43 GPS and map integration, and crime heat maps.

44 A dedicated mobile application.

45 AI-based crime prediction and analytics, and face recognition.

46 Live chat support, and court or prison management.

47 Local hosting of Chart.js for offline use; antivirus or image re-encoding of uploads.

9.5 Recommendations

For reliable long-term use, the team recommends moving from the local XAMPP environment to a production-grade server, keeping regular database backups, running periodic security reviews, load-testing the system with many reports, and testing on multiple browsers and screen sizes.

9.6 Final Remarks

The project shows that a secure and well-structured crime reporting and case management platform can be built with common web technologies within one semester. With the planned enhancements, CRMS can serve as a solid foundation for a larger digital policing service.

REFERENCES

1. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

2. Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.

3. Dennis, A., Wixom, B. H., & Roth, R. M. (2019). Systems Analysis and Design (7th ed.). Wiley.

4. Lecture Notes and Course Materials. (n.d.). Google Classroom.

5. Software Requirements Specification of Crime Reporting and Case Management System, Version 1.0.

6. Online Crime Reporting and Case Management System: Test Case Report.

7. PHP Group. PHP Manual. https://www.php.net/manual/

8. MySQL Reference Manual. https://dev.mysql.com/doc/

9. Bootstrap. Bootstrap Documentation. https://getbootstrap.com/

END OF REPORT

University of Information Technology & Sciences (UITS)
Department of Computer Science & Engineering
BSc. in CSE | CSE0613226: Software Engineering and System Analysis Lab

Skill Mapping and Justification Form

CO–PO Mapping, Complex Engineering Problem (CEP) and Knowledge Profile (KP) Justification Report

Project Title: Online Crime Reporting and Case Management System
Team: Syntax Squad

No.	COs	POs	CP/WP	CA/EA	KP/WK
CO1	Analyze requirements and model a software system using SRS and UML, and explain the fundamentals of software engineering and system analysis.	1	-	-	2, 3
CO2	By using modern tools, apply system analysis and design principles to design the system architecture and database.	2	-	-	3
CO3	Demonstrate knowledge in a team-based project by implementing and testing a working web-based system.	3	1, 3, 6, 7	-	4, 5, 6

COs – Course Outcome; POs – Program Outcome; CP/WP – Complex Problems; CA/EA – Complex Activities; KP/WK – Knowledge Profile.

Justification of Program Outcome (PO) through Course Outcome (COs)

COs	POs	Justification
CO1	PO1 (Engineering Knowledge)	We applied software engineering fundamentals by writing an SRS with 20 functional requirements and a traceability matrix (Ch. 2). We also modeled the system with use case, DFD, ER, class, sequence and state diagrams (Ch. 3, Figures 1–10).
CO2	PO2 (Problem Analysis)	We analyzed the problems of manual crime reporting for citizens, police and administrators (Ch. 1). From this we defined the report lifecycle, role permissions, validation rules and security needs (Ch. 2 and Ch. 5, Section 5.4).
CO3	PO3 (Design/Development of Solutions)	We designed and built a three-tier web system with PHP and MySQL, with Citizen, Police Officer and Administrator modules (Ch. 4–7). We tested it with 136 test cases and fixed all 5 defects found (Ch. 8).

Justification of Complex Engineering Problem (P) Attributes

Attributes	Justification
P1: Depth of Knowledge	Needs knowledge of requirements engineering, UML, database design, SQL, authentication and file upload validation (K3). Needs design of a three-tier system with role modules (K4). Needs practice with Git, PHPUnit and testing (K5). (Ch. 2–5, 8)
P3: Depth of Analysis Required	We analyzed the status lifecycle (Ch. 3, Section 3.6), set validation limits for all inputs (Ch. 2, Section 2.6) and used boundary value analysis in testing (Ch. 8). We also listed weaknesses such as no login lockout and no session timeout (Ch. 8, Section 8.7).
P6: Extent of Stakeholder Involvement and Conflicting Requirements	Three user groups have different needs: citizens want easy reporting and updates, police want an organized case queue, and administrators want control and monitoring. Each has its own portal and permissions (Ch. 1, Section 1.4; Ch. 2).
P7: Interdependence	The modules depend on each other. A report submission goes to the police queue, and every status change updates the investigations table, case status history and notifications (Ch. 3, Figure 6; Ch. 4; Ch. 6).

Justification of Knowledge Profile (KP)

Knowledge Profile Attributes	Justification
K2 – Conceptual Understanding of Engineering Sciences	We used conceptual modeling such as ER cardinality, User generalization into three roles, and mapping the ER model to database tables (Ch. 3 and Ch. 4).
K6 – Engineering Practice	We followed a structured process: requirements, modeling, implementation, testing and documentation, using traceability, Git version control and a defect log (Ch. 1, Section 1.5; Ch. 2, Section 2.7; Ch. 8).